1 Evaluation Appendix

All case study source code, specifications, and grading infrastructure are available in a single public repository (`bsc-apm-case-study-task`).[1]

1.1 Repository Organization

The repository has three top-level directories:

- `task/`:Product Requirements Document, default prompt, and scoring rubric.
- `grader/`:Black-box test suite, data processing pipeline, chat parsers, and SSO mock fixtures.
- `participant-guides/`:LaTeX participant guide for GitHub Copilot.

1.2 Baseline Reproducibility

All tests run against fixed baseline commits:

- `eclass-mcp-server` at commit `8b4e39c` (main branch)
- `openeclass` at commit `e8b3329` (Docker image `openeclass:test`)

The grading infrastructure applies participant patches against these exact versions. A submission with no patch applied receives the baseline score of 12/100, representing only the existing Python functionality. Environment setup, Docker configuration, and test data seeding are documented in `grader/SETUP.md`.

1.3 Test Suite

The test suite has three categories totaling 100 points:

- Python Baseline (12 points):11 test cases covering build, protocol compliance, login, logout, courses, and authentication status.
- Python Features (36 points):12 test cases split evenly between the two announcement tools (18 points each), covering error handling, date filtering, output format, and source aggregation.
- C Replica (52 points):28 test cases covering compilation, Valgrind memory safety, protocol compliance, and all six tools (login, logout, courses, authstatus, and both announcement tools).

Point allocations and test definitions are in `grader/src/scorer.py`; the scoring rubric is in `task/SCORING.md`.

[1] Repository available at: https://github.com/sdi2200262/bsc-apm-case-study-task

1.4 Submission Workflow

Each participant completes the task twice in isolated sessions. For each session, the participant creates a dedicated Git branch from the clean `main` baseline (e.g., `s1_apm`), works on the task using the assigned framework, then generates a solution patch as the diff between `main` and the session branch. After exporting chat logs and saving the patch, the session branch is deleted before the second session begins.

1.4.1 Submission Format

Submissions follow a fixed naming convention for automated processing:

`{PID}_{S1|S2}_{platform}_{tool}_submission.zip`

The fields are: participant ID (e.g., `P001`), session number, platform identifier (`copilot`), and framework identifier (`apm` or `spec-kit`).

Each zip contains two items:

1. A patch file (`_solution.patch`) generated via `git diff main <branch>`.
2. A logs folder (`_logs/chats/`) containing exported Copilot conversations in JSON format.

The participant guide (`participant-guide-copilot.tex`) provides full instructions for environment setup, chat export via the VS Code Command Palette, and submission packaging. *[Model strategy forthcoming.]*

1.5 Data Privacy and Anonymization

All chat exports are anonymized and encrypted during the parsing stage of the grading pipeline.

Each conversation is assigned a unique chat ID computed as the first 16 hexadecimal characters of a SHA256 hash over concatenated user messages. This ID is computed before encryption and preserved in plaintext alongside the following metadata: distinct model IDs, total request count, sanitization timestamp, and original payload size. This metadata enables dataset organization and integrity validation without requiring decryption.

Conversation content is encrypted using an HMAC-SHA256 XOR cipher. A random salt seeds keystream expansion; the serialized chat payload is XOR-encrypted against this keystream and stored in the format `ENC:<salt>:<ciphertext>`, both Base64-encoded. Decryption with the shared secret produces output bitwise-identical to the original.

1.6 Grading Pipeline

The grading pipeline has six stages with manifest-based checkpointing:

1. Validation:verifies zip structure and naming format.
2. Grading:runs the black-box test suite against Python and C implementations.
3. Parsing:extracts interaction metrics from chat exports and applies the encryption described in Section 1.5.
4. Combination:merges grading results with interaction metrics into unified result files.
5. Dataset Aggregation:appends processed data to platform-level CSV datasets.
6. Integrity Validation:verifies dataset consistency against manifest checksums.

Manifests are stored per-platform, recording checksums and timestamps at each stage. Pipeline documentation is in `grader/PIPELINE.md`.

1.7 Additional Resources

All resources live in the public repository:

Task Specification `task/`

Grading Infrastructure `grader/`

Participant Guide `participant-guides/`

Pipeline Documentation `grader/PIPELINE.md`

Setup Instructions `grader/SETUP.md`